REAL-TIME TRAIN TICKET
BOOKING PLATFORM

Google AI Studio — Full-Stack Frontend & Backend Prompt Pack

Project goal: Build a modern, responsive train discovery and ticket-booking application with live-status-ready architecture, secure accounts, booking management, and a clean passenger experience.

Important: This prompt pack is for a legitimate prototype or an authorized railway integration. Do not scrape railway websites, bypass CAPTCHA, imitate payment confirmation, or claim live seat availability unless an approved provider/API supplies it. Use clearly labelled demo data until real API credentials and permissions are available.

Prepared for use in Google AI Studio

1. How to use this prompt pack

Open Google AI Studio, start a new build/project, and provide the prompts below in sequence. Ask the model to create the project in small milestones, run the app, and fix errors before moving to the next milestone.

Recommended sequence

1. Paste the Master Project Prompt to establish the architecture and requirements.
2. Paste the Frontend Prompt to build the user interface.
3. Paste the Backend Prompt to implement APIs, database models, authentication, and booking logic.
4. Paste the Integration & Testing Prompt to connect both sides and verify end-to-end flows.

Suggested stack

Frontend: React + TypeScript + Vite, Tailwind CSS, React Router, TanStack Query, accessible component patterns.
Backend: Node.js + Express + TypeScript, PostgreSQL + Prisma ORM, Redis for short-lived caching/locking (optional), Zod validation.
Authentication: secure session or JWT with refresh-token rotation, password hashing, rate limiting.
Testing: Vitest/Jest, Supertest, Playwright.

2. Master Project Prompt

```
Act as a senior product architect, UX designer, and full-stack engineer. Build a production-minded train discovery and tic

PRODUCT NAME: RailConnect (working name; make it configurable).

PRIMARY GOAL:
Users can search trains between stations for a selected travel date, view schedules and class/fare information, check avai

CORE MODULES:
1. Public landing page with source station, destination station, journey date, passenger count, and search.
2. Station autocomplete using station code, station name, and city; prevent selecting the same origin and destination.
3. Train results with train number/name, departure/arrival, duration, intermediate stops, running days, class options, far
4. Filters and sorting: departure time, arrival time, duration, class, train type, and price.
5. Train details and timetable.
6. Availability and fare panel. Clearly distinguish LIVE provider data, cached data, and DEMO data with timestamp and sour
7. Passenger details form with age, name, gender (only if required by provider), berth preference, and contact details. Mi
8. Review journey and fare breakdown: base fare, taxes/fees, discounts, total.
9. Payment flow using a sandbox/provider adapter. Never store card number, CVV, UPI PIN, or banking passwords.
10. Booking confirmation with a unique platform booking reference and provider PNR only when returned by the authorized pr
11. My Trips: upcoming/past bookings, booking detail, cancellation request/status, downloadable ticket view.
12. Account: sign up, sign in, profile, password reset, logout.
13. Admin area protected by role-based access: manage stations/trains in demo mode, view bookings, audit events, and servi

TECHNICAL REQUIREMENTS:
- Build frontend and backend as separate folders in one repository.
- Use React + TypeScript + Vite + Tailwind on frontend.
- Use Node.js + Express + TypeScript + PostgreSQL + Prisma on backend.
- Use REST APIs with documented request/response examples.
- Use environment variables for secrets; provide .env.example, never hardcode credentials.
- Validate inputs on both client and server.
- Add database migrations and seed scripts with clearly labelled sample trains/stations.
- Add loading, empty, success, and error states.
- Add responsive mobile-first design and accessibility (keyboard support, labels, contrast).
- Add unit/integration tests for search, authentication, booking, payment callback handling, and cancellation.
- Add structured logs, request IDs, centralized error handling, rate limits, CORS allowlist, secure headers, and audit log
- Use database transactions and idempotency keys to prevent duplicate bookings/payments.
- Protect against SQL injection, XSS, CSRF where applicable, broken access control, brute force, and insecure direct objec

RAILWAY DATA / COMPLIANCE:
Create a RailwayProvider interface/adapter. Implement a DemoRailwayProvider with seeded sample data. Also provide a clearl

PAYMENT:
Create a PaymentProvider interface and a sandbox implementation. Confirm a booking only after a verified server-side payme

DELIVERABLES:
- Complete runnable source code, not pseudocode.
- Repository folder tree.
- Database schema and migrations.
- API route list and sample JSON.
- .env.example and setup instructions.
```

```
- Seed data and demo login instructions.
- Tests and commands to run them.
- README with local development and deployment steps.
- A list of items requiring real provider credentials/approval.

WORK STYLE:
First inspect requirements and propose the folder structure. Then implement in milestones: scaffold, database, backend API
```

3. Frontend Prompt

```
Build the complete FRONTEND for RailConnect using React, TypeScript, Vite, Tailwind CSS, React Router, and TanStack Query.

DESIGN DIRECTION:
Create a polished, trustworthy railway travel product. Use a clean white/light background, deep navy and railway-blue acce

PAGES / ROUTES:
- / : Landing page with header, hero search card, popular routes (sample labels), service highlights, footer.
- /trains : Search results; preserve search query in URL.
- /trains/:trainId : Train details, route timeline, timetable, class/fare/availability.
- /booking/:trainId : Passenger details and journey review.
- /payment/:bookingId : Payment status and sandbox checkout.
- /booking-confirmation/:bookingId : Ticket summary and status.
- /login, /register, /forgot-password.
- /account/profile.
- /my-trips and /my-trips/:bookingId.
- /admin : role-protected demo admin dashboard.

SEARCH UX:
Use accessible station autocomplete with keyboard navigation. Fields: From, To, Date, passengers, and Search Trains. Swap

RESULTS UX:
Show train number/name, departure and arrival stations/times, duration, stops, running days, class tabs, fare, and availab

BOOKING UX:
Build a stepper: Select train → Passenger details → Review → Payment → Confirmation. Validate passenger fields. Display

ACCOUNT UX:
Implement login/register forms, session handling, protected routes, logout, profile, My Trips, booking detail, and cancell

COMPONENTS:
Header, Footer, SearchForm, StationAutocomplete, DateField, PassengerSelector, TrainCard, TrainFilters, TrainTimeline, Cla

ENGINEERING:
- Create typed API client and TypeScript interfaces matching backend contracts.
- Use environment variable VITE_API_BASE_URL.
- Use TanStack Query for server state, caching, retries, and invalidation.
- Do not store passwords or payment secrets in localStorage.
- Handle 401/403/404/409/422/429/500 responses.
- Add accessible labels, focus states, keyboard support, and semantic HTML.
- Add responsive layouts for 360px mobile, tablet, and desktop.
- Add component tests and Playwright flows for search and demo booking.
- Include .env.example and README.

IMPORTANT:
Use seeded demo data if backend provider is in demo mode. Display a visible “DEMO — not live railway data” notice in all r

Implement the actual files and working code. Start by showing the folder structure, then build the pages and components, c
```

4. Backend Prompt

```
Build the complete BACKEND for RailConnect using Node.js, Express, TypeScript, PostgreSQL, and Prisma ORM. Create a secure

ARCHITECTURE:
Use a modular structure: src/config, src/middleware, src/modules/auth, users, stations, trains, availability, bookings, pa

DATABASE MODELS:
- User: id, name, email (unique), phone (optional), passwordHash, role, createdAt, updatedAt.
- Station: id, code (unique), name, city, state, active.
- Train: id, trainNumber (unique), name, trainType, active.
- TrainStop: id, trainId, stationId, sequence, arrivalTime, departureTime, dayOffset.
- TrainClass: id, trainId, code, label.
- FareQuote: id, trainId, classCode, amount, currency, sourceMode, fetchedAt, expiresAt.
- AvailabilitySnapshot: id, trainId, journeyDate, classCode, status, availableCount (nullable), sourceMode, fetchedAt.
- Booking: id, publicReference (unique), userId, trainId, journeyDate, classCode, status, totalAmount, currency, idempoten
- Passenger: id, bookingId, name, age, gender (nullable), berthPreference (nullable).
- Payment: id, bookingId, provider, providerPaymentId (nullable), amount, currency, status, idempotencyKey, paidAt (nullab
- AuditLog: id, actorUserId (nullable), action, resourceType, resourceId, requestId, createdAt.

API ROUTES:
GET /api/health
GET /api/stations?query=
GET /api/trains/search?from=&to=&date=&passengers=
GET /api/trains/:trainId
GET /api/trains/:trainId/availability?date=&class=
POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
POST /api/auth/forgot-password
GET /api/users/me
PATCH /api/users/me
POST /api/bookings/quote
POST /api/bookings
GET /api/bookings
GET /api/bookings/:bookingId
POST /api/bookings/:bookingId/cancel
POST /api/payments/create-session
POST /api/payments/webhook
GET /api/admin/overview
GET /api/admin/bookings

BUSINESS LOGIC:
- Validate station codes, journey date, passenger count, train route, class, and booking ownership.
- Search only trains whose stop sequence includes origin before destination.
- Calculate duration across day boundaries correctly.
- Use provider adapters for schedule, fare, and availability.
- Store sourceMode = DEMO, CACHED, or LIVE and fetchedAt on returned availability/fare.
- Never treat stale cache or demo results as live availability.
- Create booking using a database transaction and idempotency key.
- Re-check provider availability immediately before booking if authorized provider supports it.
- Do not oversell: use database locking/unique constraints or provider confirmation flow as appropriate.
- Create booking as PENDING_PAYMENT; transition to CONFIRMED only after verified payment and required provider confirmatio
- Cancellation must check policy and provider result; use REQUESTED/PROCESSING/CANCELLED/REJECTED states.
- Never generate or invent an official railway PNR. providerPnr stays null until supplied by authorized provider.

SECURITY:
- Hash passwords with Argon2id or bcrypt using safe parameters.
- Use secure, HttpOnly, SameSite cookies for refresh/session tokens where appropriate.
- Validate all request bodies/query params with Zod.
- Add rate limiting to auth, search, booking, and payment endpoints.
- Use Helmet, strict CORS allowlist, centralized errors, request IDs, structured logs, and audit logs.
- Enforce authorization and booking ownership on every private route.
- Use parameterized Prisma queries.
- Avoid logging passwords, tokens, payment data, or full passenger personal data.
- Add CSRF protection if cookie-based auth is used.
- Verify payment webhook signatures and make webhook processing idempotent.
- Keep secrets only in environment variables.

PROVIDER ADAPTERS:
Define RailwayProvider methods: searchTrains, getTrainDetails, getAvailability, getFare, createReservation, cancelReservat
Define PaymentProvider methods: createCheckoutSession, verifyWebhook, refund. Implement a sandbox provider and document ho
```

```
DELIVERABLES:
- Complete TypeScript source code.
- Prisma schema, migrations, seed script.
- .env.example with placeholders only.
- API documentation with request/response JSON examples.
- Unit and integration tests for route search, auth, booking idempotency, payment webhook, and cancellation.
- README with install, database setup, migrate, seed, dev, test, and production build commands.
- Docker Compose for local PostgreSQL (and Redis only if actually used).
- Explain all required environment variables and external approvals.

Implement runnable code, not pseudocode. First create the folder structure, then implement database and providers, then AP
```

5. Integration, testing & deployment prompt

```
Now integrate the RailConnect frontend and backend into a working full-stack application.

1. Inspect both folders and compare frontend API types with backend request/response schemas. Resolve mismatches rather th
2. Configure local environment variables and CORS. Keep secrets out of frontend variables.
3. Start PostgreSQL, run Prisma migrations, seed demo stations/trains, then start backend and frontend.
4. Verify station autocomplete, train search, train details, demo availability labels, registration/login, passenger form,
5. Test invalid station pairs, past dates, no results, unavailable class, expired token, unauthorized booking access, dupl
6. Add loading/error/empty states and useful server logs. Do not expose stack traces to users.
7. Run lint, typecheck, unit tests, integration tests, and production builds. Fix all failures.
8. Provide exact Windows Command Prompt and macOS/Linux commands to install and run the project.
9. Provide deployment instructions for frontend, backend, PostgreSQL, environment variables, HTTPS, database migrations, b
10. Clearly list what works in demo mode and what remains blocked until an authorized railway data/booking provider and re

Do not claim that the platform can issue actual railway tickets until the authorized provider integration has been approve
```

6. Suggested repository structure

```
railconnect/
  frontend/
    src/
      components/
      pages/
      hooks/
      services/
      types/
      routes/
      styles/
    public/
    .env.example
    package.json
  backend/
    prisma/
      schema.prisma
      seed.ts
    src/
      config/
      middleware/
      modules/
      providers/
      utils/
      app.ts
      server.ts
    tests/
    .env.example
    package.json
  README.md
  docker-compose.yml
```

7. Final checks before a real launch

A real ticket-booking product requires a railway data/booking provider that explicitly authorizes train search, seat availability, fare quotation, reservation, PNR retrieval, and cancellation. A public timetable feed alone is not permission to book. Confirm commercial terms, security requirements, branding rules, and production access with the provider. Use payment-provider-hosted checkout and follow applicable privacy, consumer-protection, tax, and payment rules.

Do not collect or store card details, CVV, UPI PIN, or banking passwords. Do not display demo inventory as live inventory. Make booking status and refund/cancellation status clear to the passenger.

Tip: If Google AI Studio generates only a UI or stops midway, ask it: “Continue from the current repository state. Do not rewrite completed files. Implement the next incomplete milestone, run the build/tests, and fix errors.”